PRÁCTICAS DE IPO

Yoel Arroyo
Iván González
Ana Isabel Molina
Carmen Lacave

Estilos de interacción

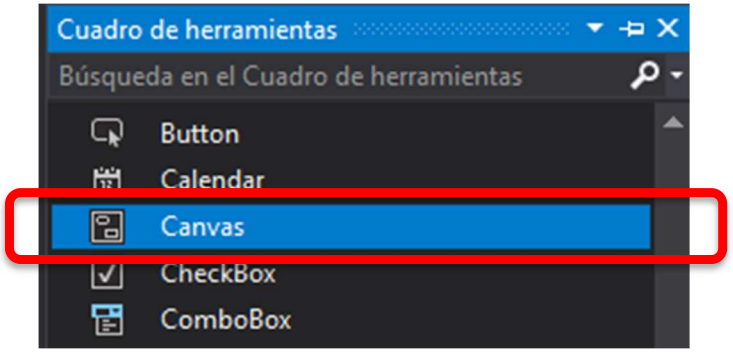
Manipulación directa

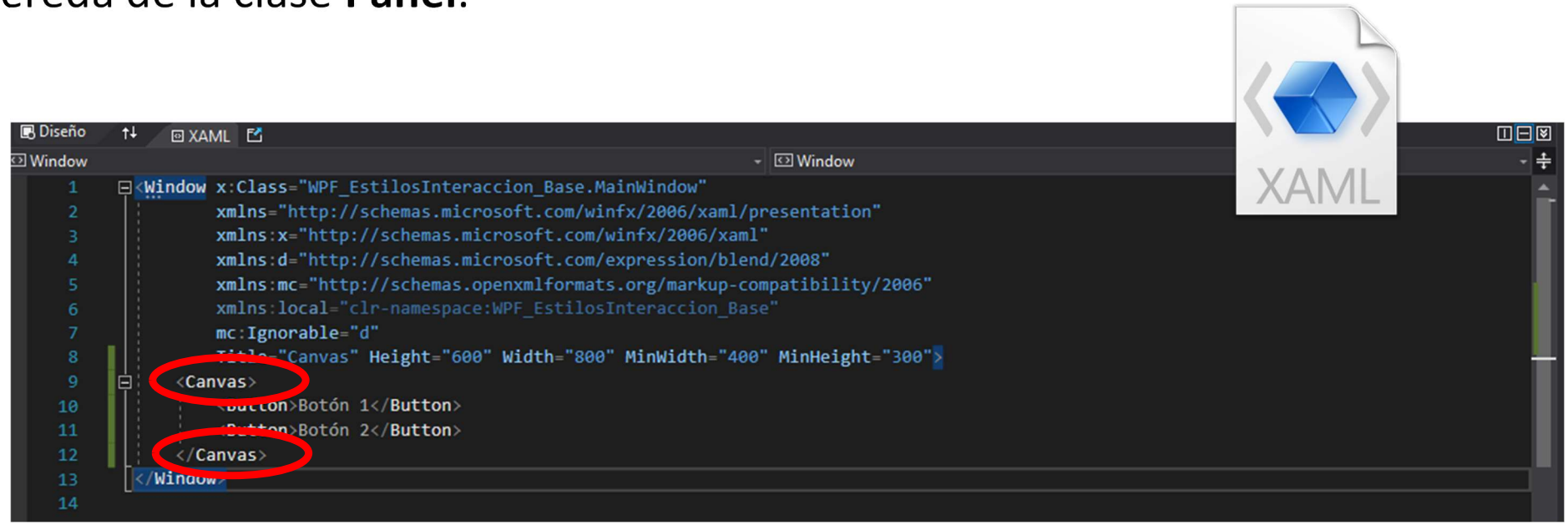
En esta sesión aprenderemos…

- Editor de dibujo (Canvas):
  - Añadido de elementos
  - Posicionamiento de elementos
  - Manejo de los ejes
  - Creación de un mapa
- Estilo de Interacción Drag & Drop:
  - Fundamentos
  - Transferencia de datos
  - Efectos gráficos
  - Eventos
  - Ampliación mapa Interactivo

Editor de dibujo: Fundamentos

- El **editor de dibujo** o **canvas** define un área dentro de la cual se pueden posicionar explícitamente los elementos hijos mediante **coordenadas relativas al área del lienzo**.
- Es uno de los sistemas de *Layout* disponibles en WPF.
- Al igual que el resto de sistemas de *Layout*, hereda de la clase **Panel**.





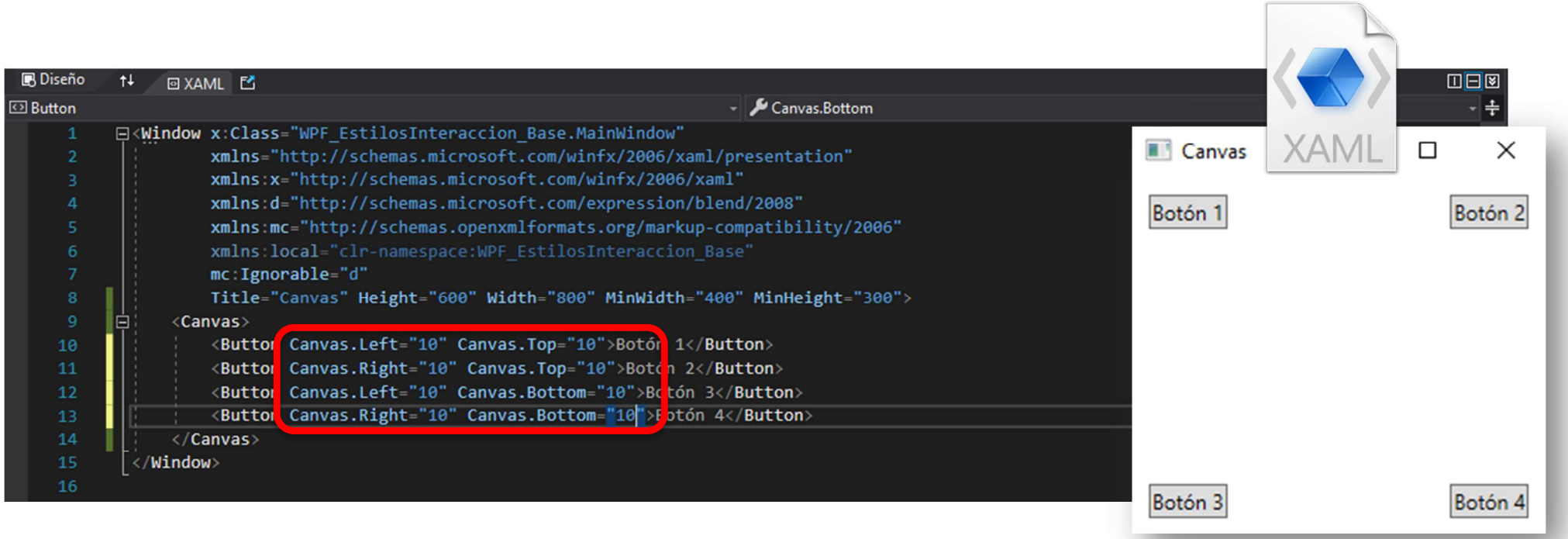
Editor de dibujo: Fundamentos

Ejemplo de Canvas sencillo con dos botones

```
<Window x:Class="WPF_EstilosInteraccion_Base.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:WPF_EstilosInteraccion_Base"
        mc:Ignorable="d"
    <Canvas>
        <Button>Botón 1</Button>
        <Button>Botón 2</Button>
    </Canvas>
</Window>
```

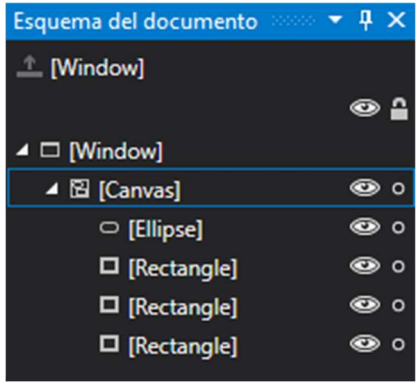
Editor de dibujo: Posicionado

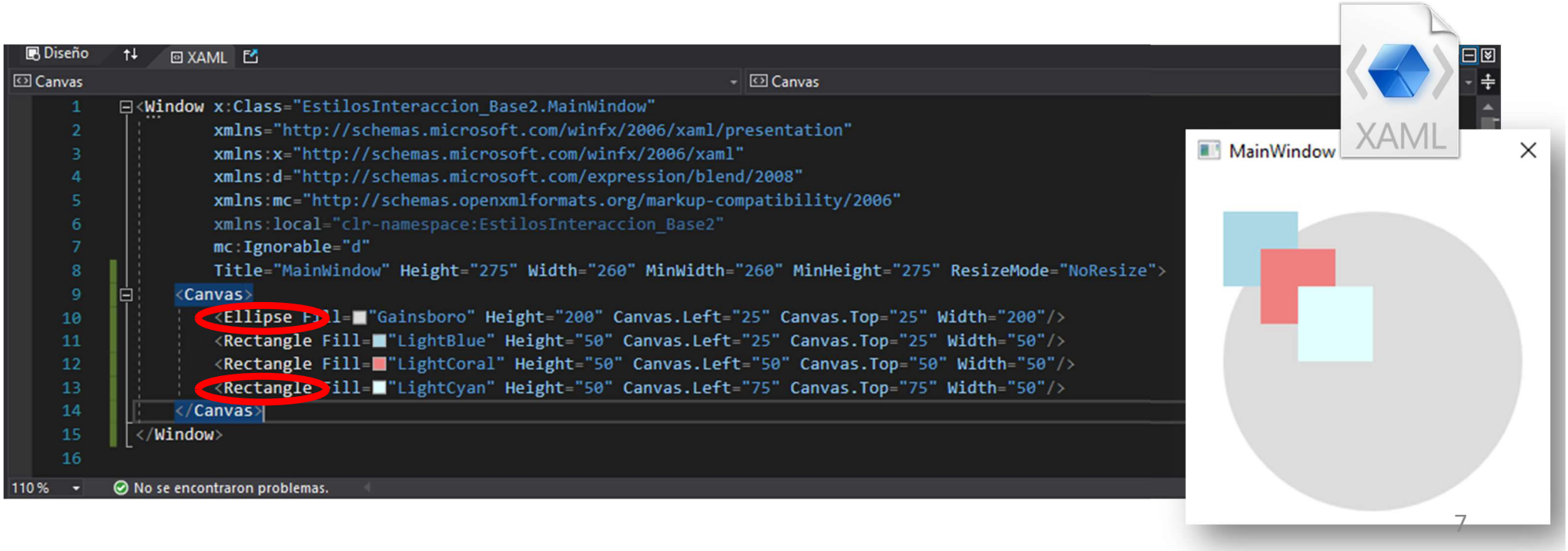
- El **canvas** es probablemente el panel más simple de todos.
- Permite añadir controles y posicionarlos usando coordenadas explícitas.
- En el ejemplo anterior sólo es visible el último botón, debido a que no se ha indicado la posición de cada uno de ellos.
- Por ejemplo: para añadir 4 botones, uno en cada esquina del lienzo, podemos arrastrar los botones o indicar las coordenadas exactas en el fichero XAML.



Editor de dibujo: Elementos y formas

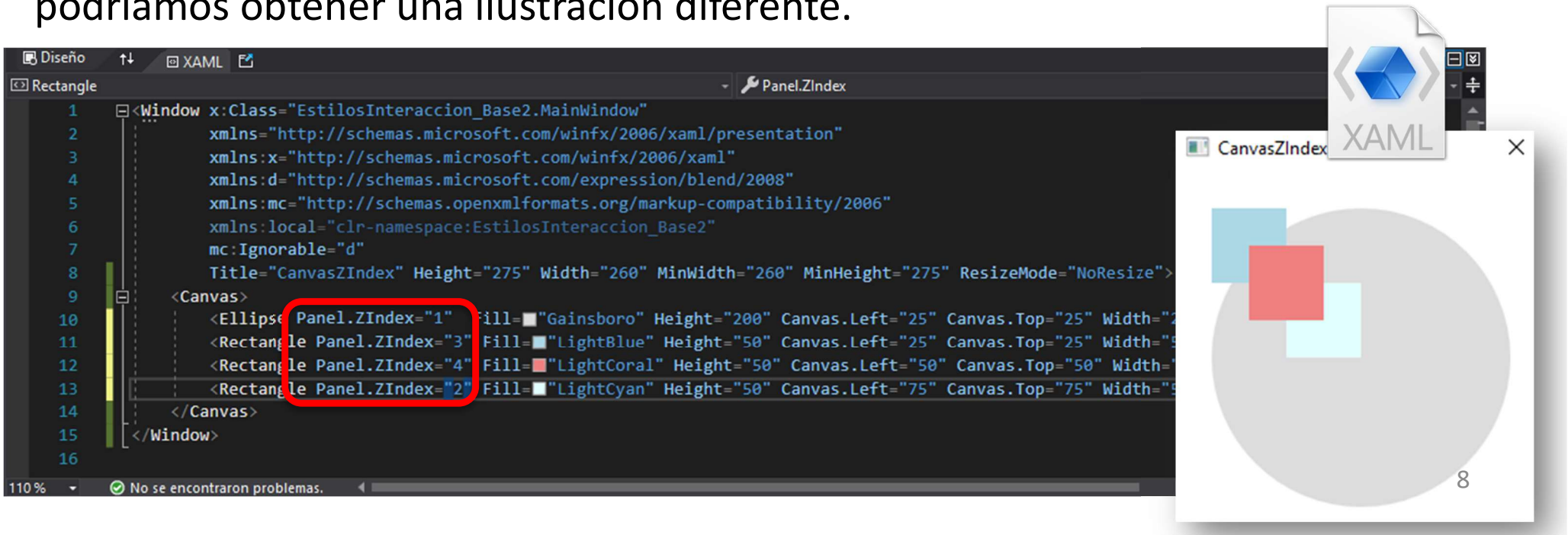
- El **canvas** proporciona un completo control sobre la posición de sus elementos, sin importar si se exceden los límites del lienzo o si algún elemento pudiera estar encima de otro.
- Esto último lo convierte en una mala elección para casi cualquier tipo de diseño de diálogo. Sin embargo, es ideal para pintar.
- WPF incorpora una serie de controles que permite colocar dentro del lienzo para así hacer ilustraciones personalizadas.





Editor de dibujo: Eje Z

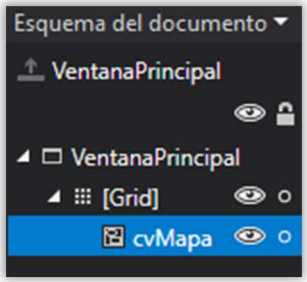
- Un concepto importante al utilizar canvas es el manejo del **eje Z**.
- Si dos controles dentro de un canvas se superponen, el último definido tendrá prioridad sobre el resto.
- Una propiedad que incorpora la clase *Panel*, y que permite personalizar el orden de los elementos, es **ZIndex**. El valor por defecto es 0, y a mayor número mayor será la prioridad del elemento sobre el resto.
- Por ejemplo: si modificáramos el eje Z de los elementos del ejemplo anterior podríamos obtener una ilustración diferente.

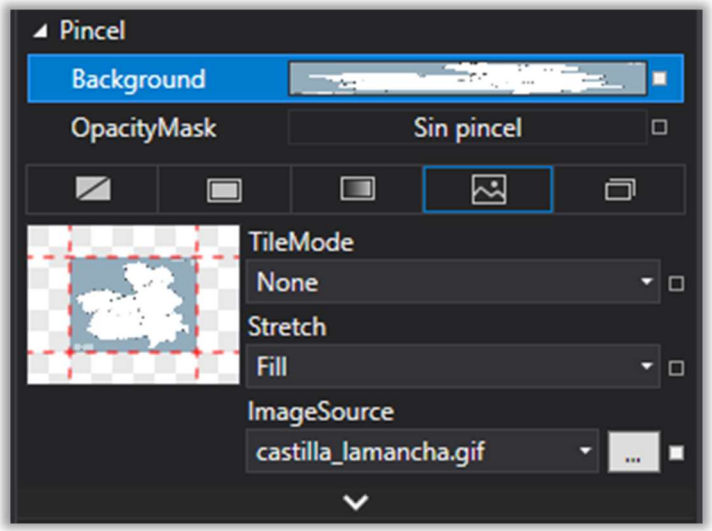


Editor de dibujo: Mapa

- Cambiar propiedades de la GUI
  - **Window**:

```
<Window ... Title="Mapa VS Code"
Height="713" Width="766" MinWidth="383"
MinHeight="354" ResizeMode="NoResize">
```





  - Creamos carpeta “images”:
    - Explorador de soluciones > Agregar > Nueva Carpeta: “images”
    - images > Agregar elemento existente: “Añadir las imágenes de recursos”
  - Añadimos y ajustamos **Canvas** al Grid:

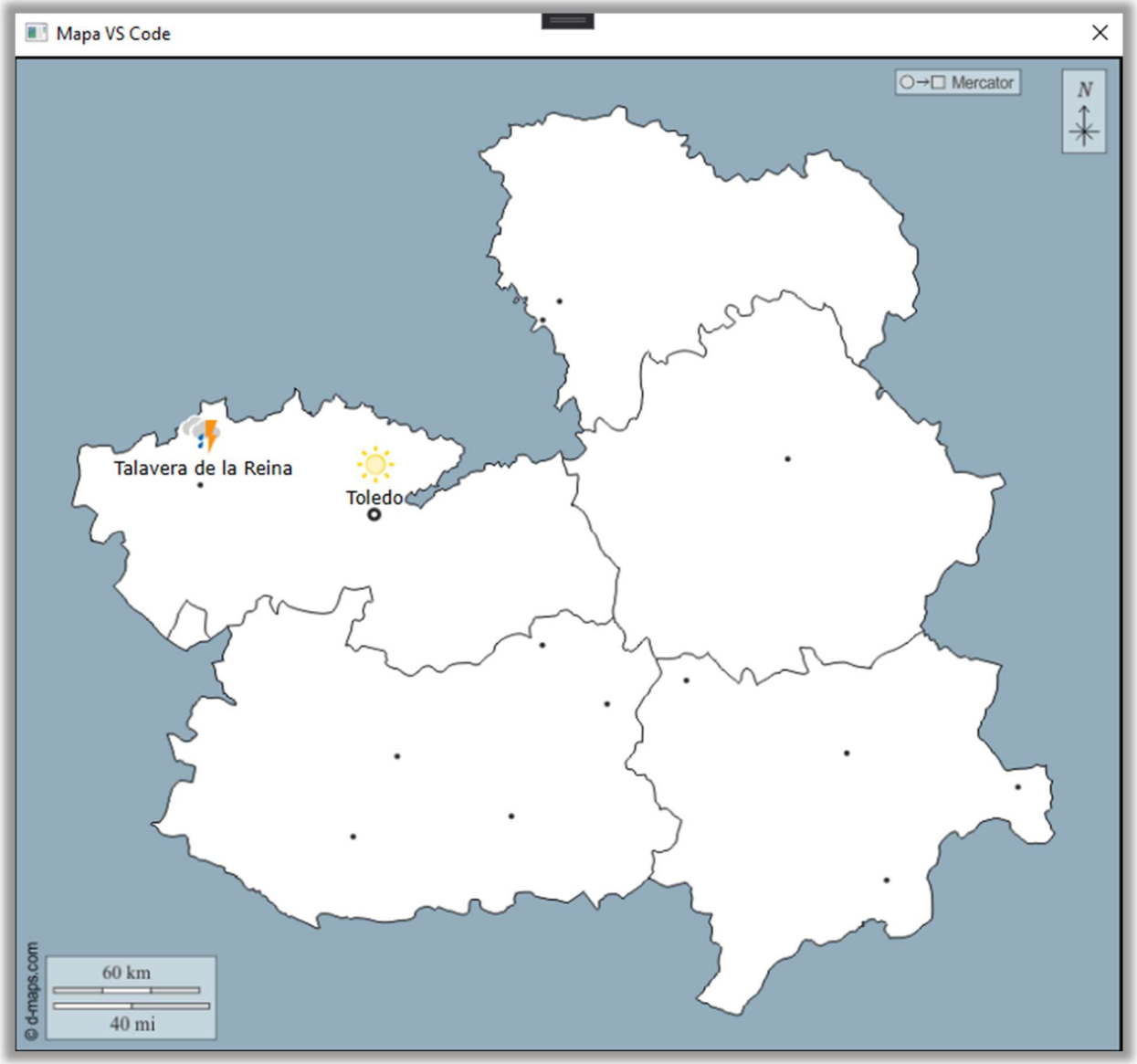
```
<Canvas x:Name="cvMapa">
    <Canvas.Background>
        <ImageBrush ImageSource="images/castilla_lamancha.gif"/>
    </Canvas.Background>
</Canvas>
```

Editor de dibujo: Mapa

- Cambiar propiedades de la GUI
  - Añadimos **Labels** e **Images**:

```
<Canvas x:Name="cvMapa">
      <Canvas.Background>
            <ImageBrush ImageSource="images/castilla_lamancha.gif"/>
      </Canvas.Background>
      <Label x:Name="lblTalavera" Content="Talavera de la Reina" Canvas.Top="267" Canvas.Left="75"/>
      <Image x:Name="imgTalavera" Canvas.Left="120" Source="images/tormenta.png" Width="24" Height="24"
Canvas.Top="253"/>
      <Label x:Name="lblToledo" Content="Toledo" Canvas.Top="286" Canvas.Left="224"/>
      <Image x:Name="imgToledo" Canvas.Left="237" Source="images/despejado.png" Width="24" Height="24“
Canvas.Top="270"/>
</Canvas>
```

Resultado. Propuesta de mejora



- Completar el mapa añadiendo las ciudades principales
- Añadir temperaturas máximas / mínimas para las capitales

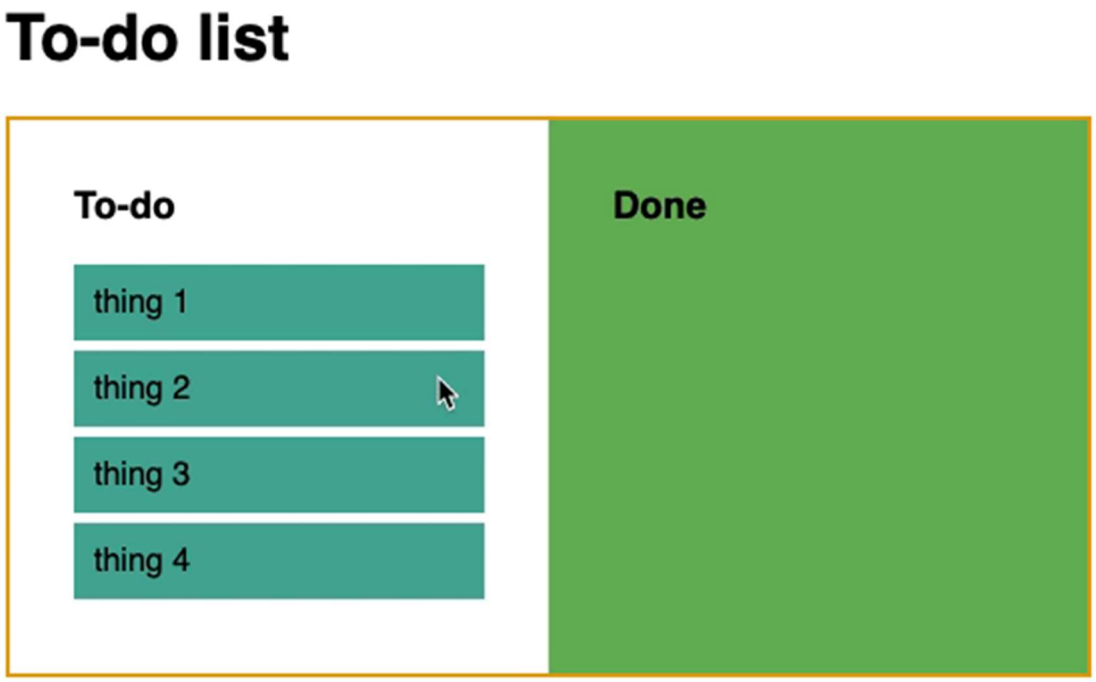
WPF
Microsoft .NET
Windows Presentation Foundation

Arrastrar y soltar: Fundamentos

- Cualquier **UIElement** o **ContentElement** puede participar en una operación de arrastrar y soltar.
- Los eventos y métodos necesarios para las operaciones de arrastrar y soltar se definen en la clase *DragDrop*.
- En las operaciones arrastrar y soltar suelen participar dos partes: un **origen de arrastre**, donde se origina la acción, y un **destino de colocación**, que recibe el objeto al colocarlo.
- El **origen de arrastre** y el **destino de colocación** pueden ser elementos de interfaz de usuario de la misma aplicación u otra distinta.
- El tipo y número de objetos que se pueden manipular con arrastrar y colocar son completamente arbitrarios. Por ejemplo: archivos, carpetas y elementos de la interfaz.

Arrastrar y soltar: Transferencia de datos

- Arrastrar y soltar forma parte del área más general de la transferencia de datos. Este tipo de operación requiere:
  - Un objeto de origen que proporcione los datos.
  - Una manera de almacenar temporalmente los datos transferidos.
  - Un objeto de destino que reciba los datos.
- En una operación de arrastrar y soltar se utiliza un ***DataObject*** para almacenar los datos.



Efectos gráficos de las operaciones arrastrar-soltar

- WPF define una enumeración ***DragDropEffects*** que puede utilizar para especificar el efecto de una operación de arrastrar y soltar:

	Miembro	Descripción
	None	El destino (soltar) no acepta los datos.
	Copy	Los datos se copian en el destino (soltar).
	Move	Los datos del origen (arrastrar) se mueven al destino (soltar).
	Link	Los datos del origen (arrastrar) se vinculan al destino (soltar).
	Scroll	El destino se puede desplazar mientras arrastra para buscar un destino donde arrastrar que no está actualmente visible.
	All	Se copian los datos, se quitan del origen (soltar) y se colocan en el destino pertinente.

Eventos de arrastrar-soltar

- Las operaciones de arrastrar y soltar admiten un modelo basado en eventos. Tanto el origen de arrastre como el destino de colocación utilizan un conjunto estándar de eventos para controlar las operaciones de arrastrar y soltar. Los eventos asociados a soltar son:

	Evento	Resumen
	[DragEnter]	Este evento se produce cuando se arrastra un objeto a los límites del destino de colocación. Se trata de un evento de propagación.
	[DragLeave]	Este evento se produce cuando se arrastra un objeto fuera de los límites del destino de colocación.
	[DragOver]	Este evento se produce continuamente mientras se arrastra (mueve) un objeto dentro del límite del destino de colocación. Se trata de un evento de propagación.
	[Drop]	Este evento se produce cuando un objeto se coloca en el destino de colocación. Se trata de un evento de propagación.

Implementación de arrastrar-soltar

- El proceso para habilitar una interacción arrastrar-soltar es el siguiente:
  1. Identifique el elemento que será el origen de arrastre. Un origen de arrastre puede ser un **UIElement** o un **ContentElement**.
  2. Cree un controlador de eventos en el origen de arrastre que iniciará la operación de arrastrar y soltar.
  3. En el controlador de eventos del origen de arrastre, llame al método *DoDragDrop* para iniciar la operación de arrastrar y soltar. En la llamada a *DoDragDrop*, especifique el origen de arrastre, los datos que se van a transferir y los efectos permitidos.
  4. Identifique el elemento que será el destino de colocación. Un destino de colocación puede ser un **UIElement** o un **ContentElement**.
  5. En el destino de colocación, establezca la propiedad *AllowDrop* en true.
  6. En el destino de colocación, cree un controlador de eventos *Drop* para procesar los datos colocados.
  7. En el controlador de eventos *Drop*, extraiga los datos de *DragEventArgs* mediante los métodos *GetDataPresent* y *GetData*. Utilice los datos como desee.

Ejemplo: Arrastrar-soltar

- Cambiar propiedades de la GUI
  - Redimensionar **Window**: Nuevo Tamaño: 832x713
  - Crear columnas **Grid**:

```
<Grid.ColumnDefinitions>
    <ColumnDefinition Width="30*"/>
    <ColumnDefinition Width="383*"/>
</Grid.ColumnDefinitions>
```

  - Ajustamos **Canvas** a columna 1 del Grid:

```
<Canvas x:Name="cvMapa" Grid.Column="1">
```

  - Añadimos y ajustamos **StackPanel** a columna 0 del Grid:

```
<StackPanel x:Name="stpPaleta" Grid.Column="0" Cursor="Hand">
    <StackPanel.Background>
        <LinearGradientBrush EndPoint="0.5,1" StartPoint="0.5,0">
            <GradientStop Color="Black" Offset="1"/>
            <GradientStop Color="#FF979797" Offset="0.493"/>
            <GradientStop Color="Black"/>
        </LinearGradientBrush>
    </StackPanel.Background>
</StackPanel>
```

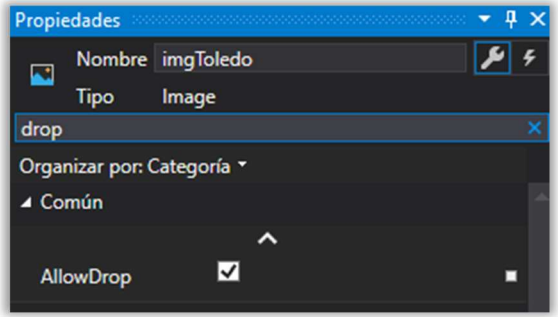
Ejemplo: Arrastrar-soltar

- Cambiar propiedades de la GUI
  - Añadimos imágenes a **StackPanel:**

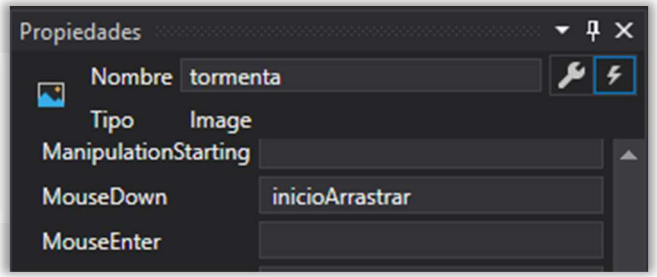
```
<StackPanel x:Name="stpPaleta" Grid.Column="0" Cursor="Hand">
    <StackPanel.Background>
        <LinearGradientBrush EndPoint="0.5,1" StartPoint="0.5,0">
            <GradientStop Color="Black" Offset="1"/>
            <GradientStop Color="#FF979797" Offset="0.493"/>
            <GradientStop Color="Black"/>
        </LinearGradientBrush>
    </StackPanel.Background>
    <Image x:Name="despejado" Margin="5" Source="images/despejado.png"/>
    <Image x:Name="nuboso" Margin="5" Source="images/nuboso.png"/>
    <Image x:Name="muynuboso" Margin="5" Source="images/muynuboso.png"/>
    <Image x:Name="niebla" Margin="5" Source="images/niebla.png"/>
    <Image x:Name="llovizna" Margin="5" Source="images/llovizna.png"/>
    <Image x:Name="lluvioso" Margin="5" Source="images/lluvioso.png"/>
    <Image x:Name="tormenta" Margin="5" Source="images/tormenta.png"/>
</StackPanel>
```

Ejemplo: Arrastrar-soltar

- Añadir funcionalidad Drag & Drop:
  1. Habilitar propiedad **AllowDrop** (*Común*) sobre imagen destino del mapa.
  2. Crear un evento **MouseDown** "*inicioArrastrar*" en imagen origen de la paleta.



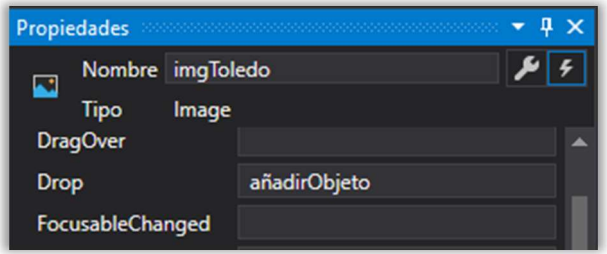

```
private void inicioArrastrar(object sender, MouseButtonEventArgs e)
{
      DataObject dataO = new DataObject((Image)sender);
      DragDrop.DoDragDrop((Image)sender, dataO, DragDropEffects.Move);
}
```



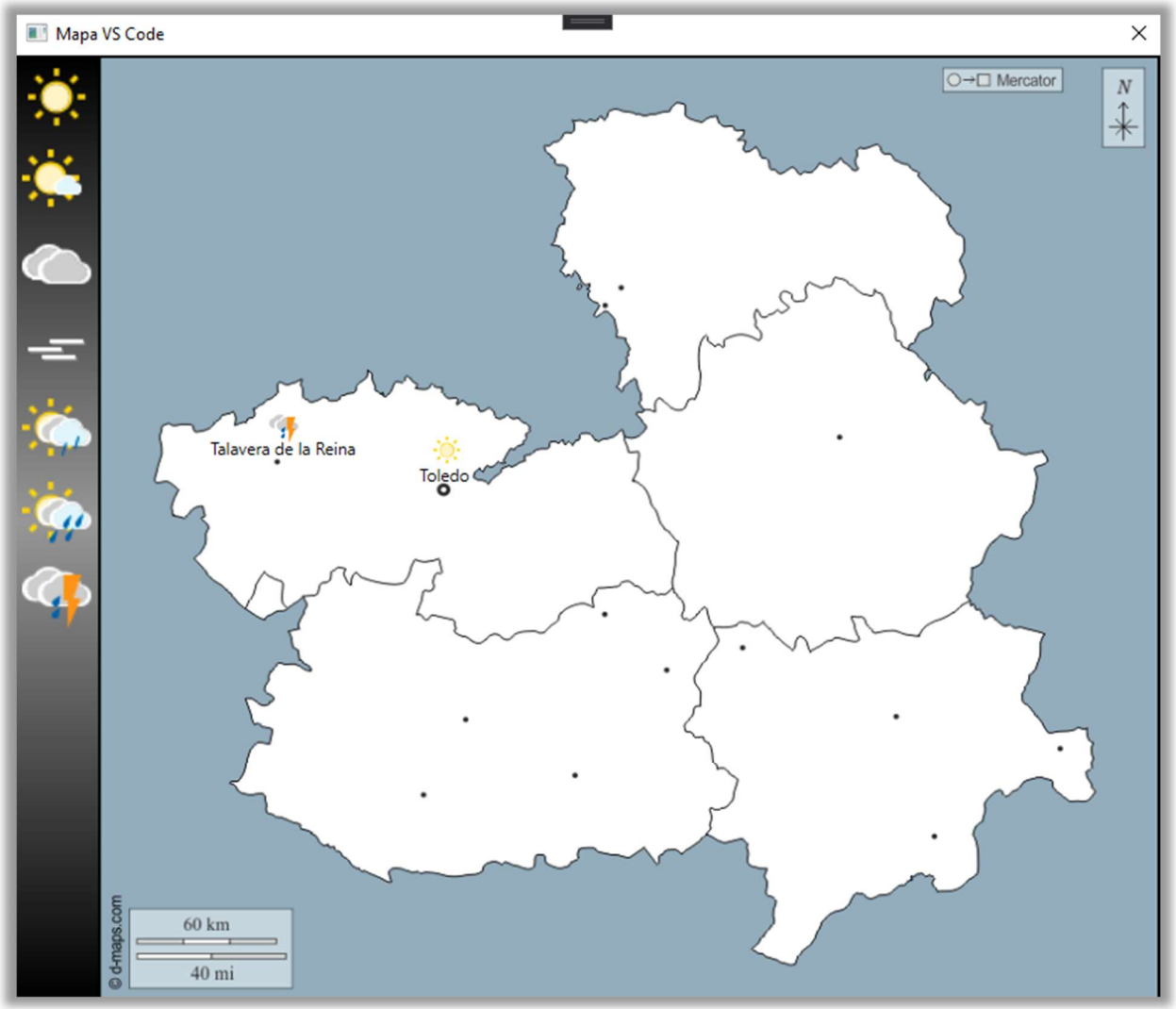
  3. Crear un evento **Drop** "*añadirObjeto*" en imagen destino del mapa.

```
private void añadirObjeto(object sender, DragEventArgs e)
{
      Image imgDragged = (Image)e.Data.GetData(typeof(Image));
      Image imgToUpdate = (Image)e.OriginalSource;

      imgToUpdate.Source = imgDragged.Source;
}
```



Resultado. Propuesta de mejora



- Dotar de funcionalidad todas las imágenes de la paleta
- Añadir una segunda paleta que permita arrastrar temperaturas sobre las capitales de provincia

WPF
Microsoft .NET
Windows Presentation Foundation